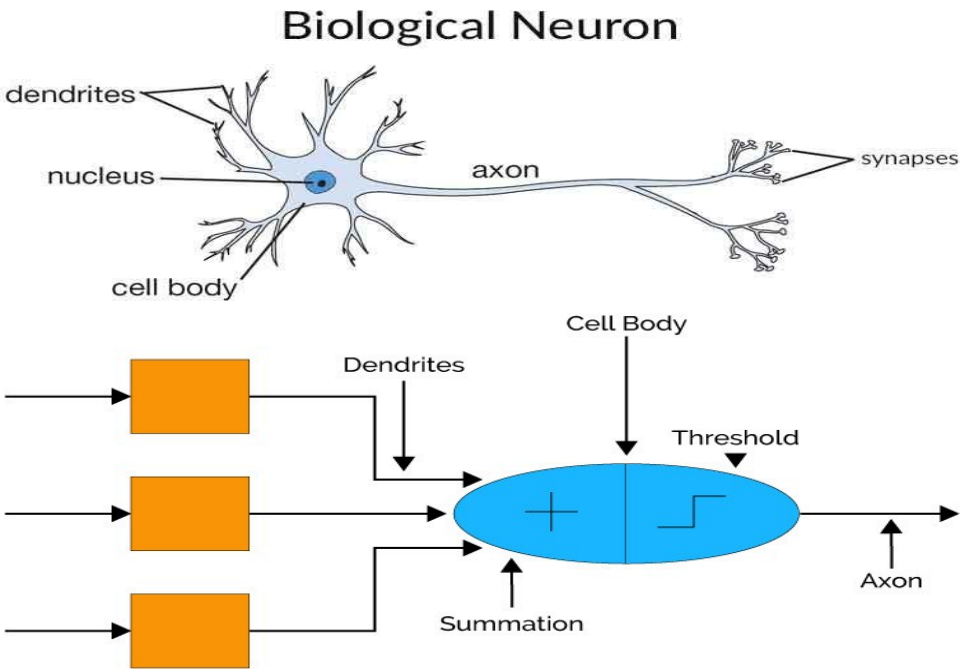
Recap: Learning and Trees

- Induction vs Deduction
- Decision trees: highly expressive hyp. space
- Learning as search.
- Numerical data: How to search in a continuous problem? → discretization

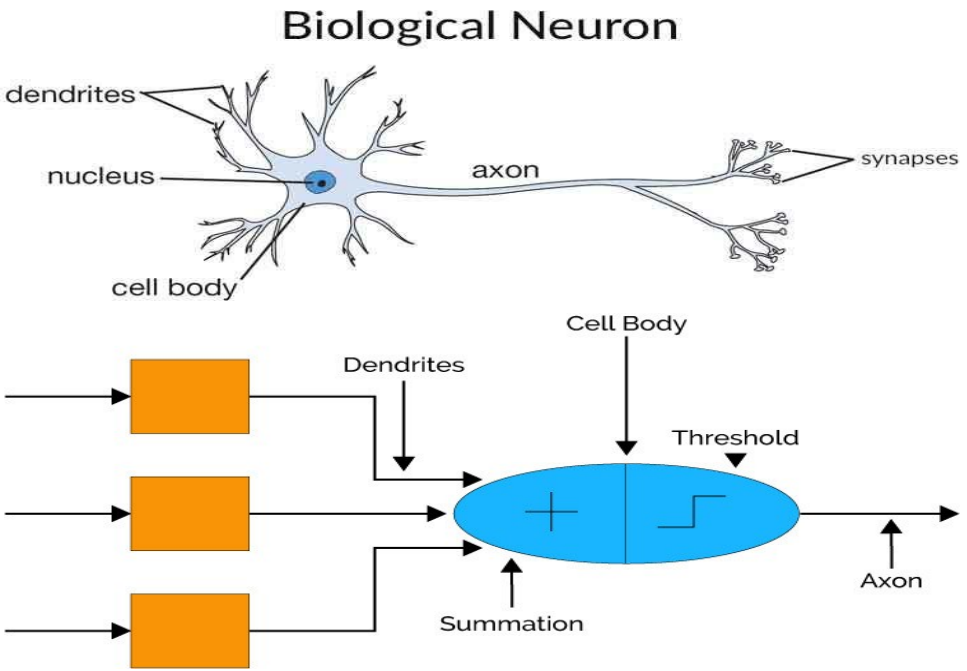
Artificial Neural Networks

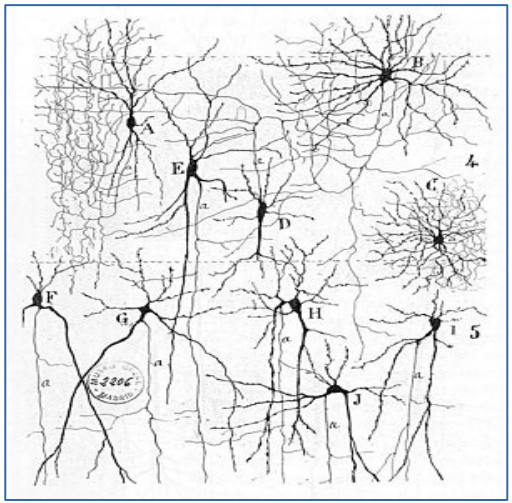
- Robust approach to approximating real and discrete-valued target functions
- Biological Motivations
  - Using ANNs to model and study biological learning processes
  - Obtaining highly effective Machine Learning algorithms by mirroring biological processes

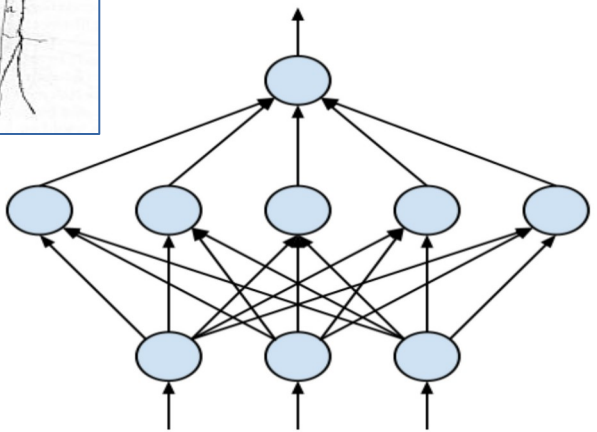
Biological motivations



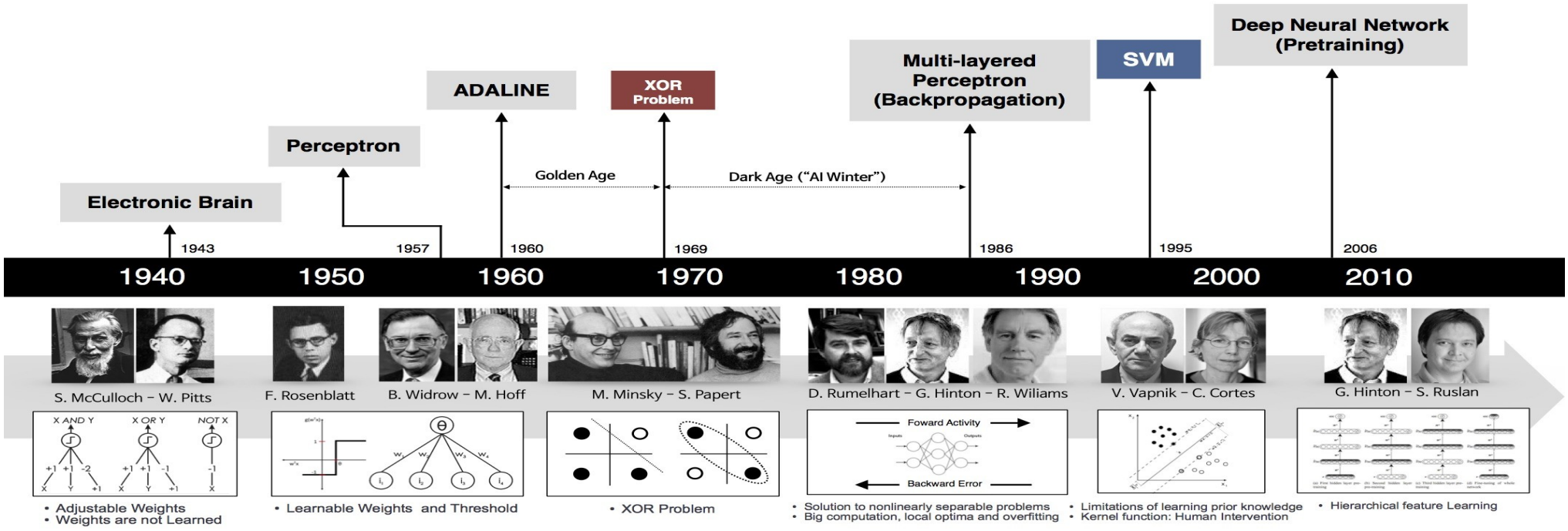
Biological motivations







Timeline



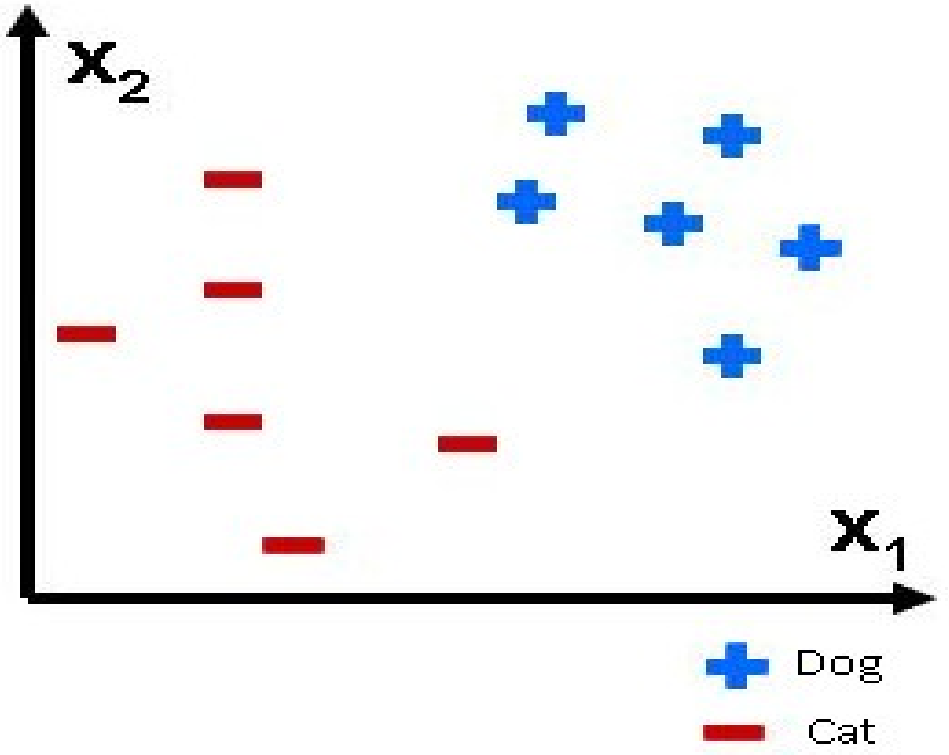
2018: Turing Award to developers of Deep Neural Networks

2024: Nobel prize (physics) to Geoff Hinton

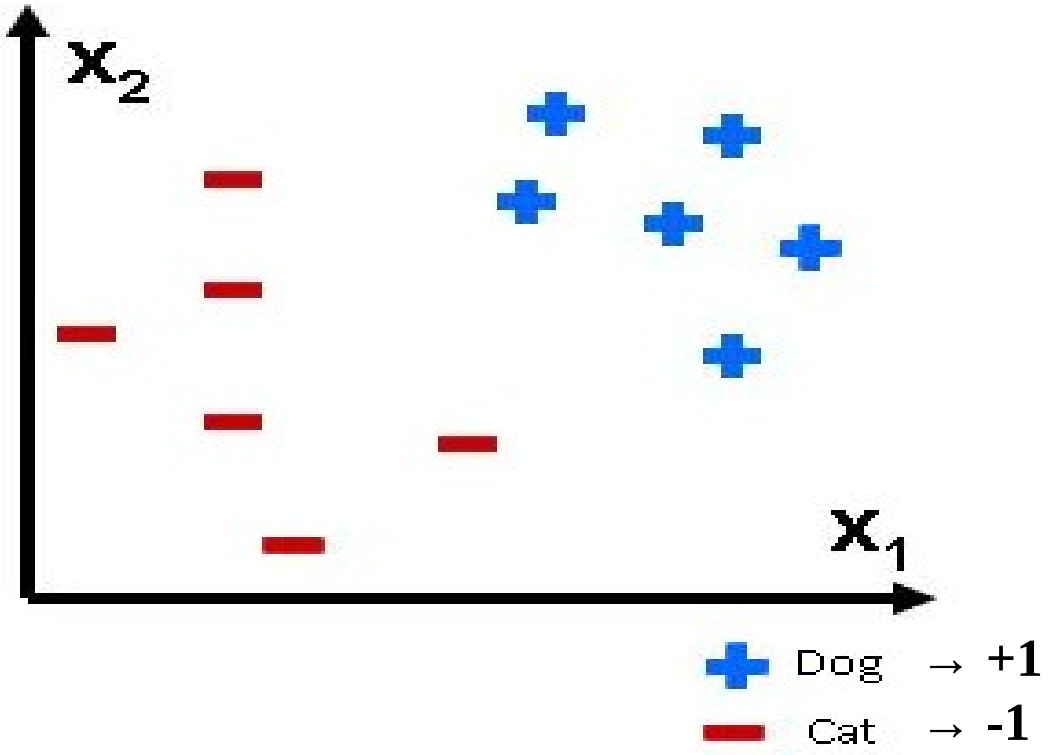
Appropriate Problems for ANNs

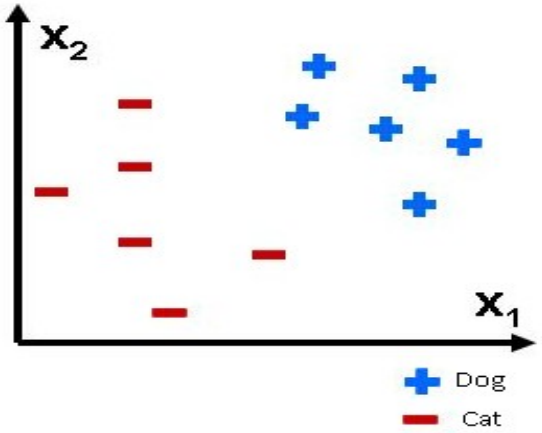
- Regression and Multiclass classification
- High dimension inputs
- Training examples may contain errors
- Long training times are acceptable
- Fast evaluation of the learned target function may be required
- Ability to understand the target function not important

New problem



New problem

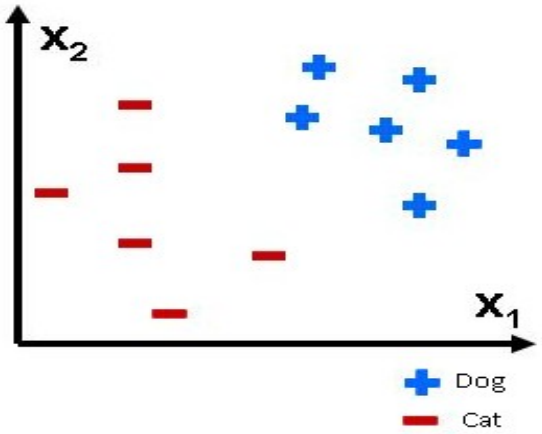




Perceptrons

$$o(x_1,x_2\ldots,x_n) = \begin{cases} 1 \quad \text{if } w_0 + w_1 x_1 + \ldots + w_n x_n > 0 \\ -1 \text{ otherwise} \end{cases}$$

$$o(\mathbf{x}) = \text{sgn}(\mathbf{w}.\mathbf{x}) \qquad (i=0 \text{ to } n \; ; \; x_0=1)$$

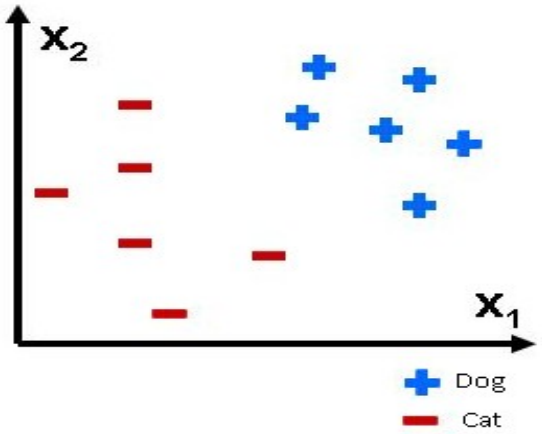


Perceptrons

$$o(x_1,x_2\ldots,x_n) = \begin{cases} 1 \quad \text{if } w_0 + w_1 x_1 + \ldots + w_n x_n > 0 \\ -1 \text{ otherwise} \end{cases}$$

$$o(\mathbf{x}) = \text{sgn}(\mathbf{w}.\mathbf{x}) \qquad (i=0 \text{ to } n \; ; \; x_0=1)$$

Hypothesis Space:



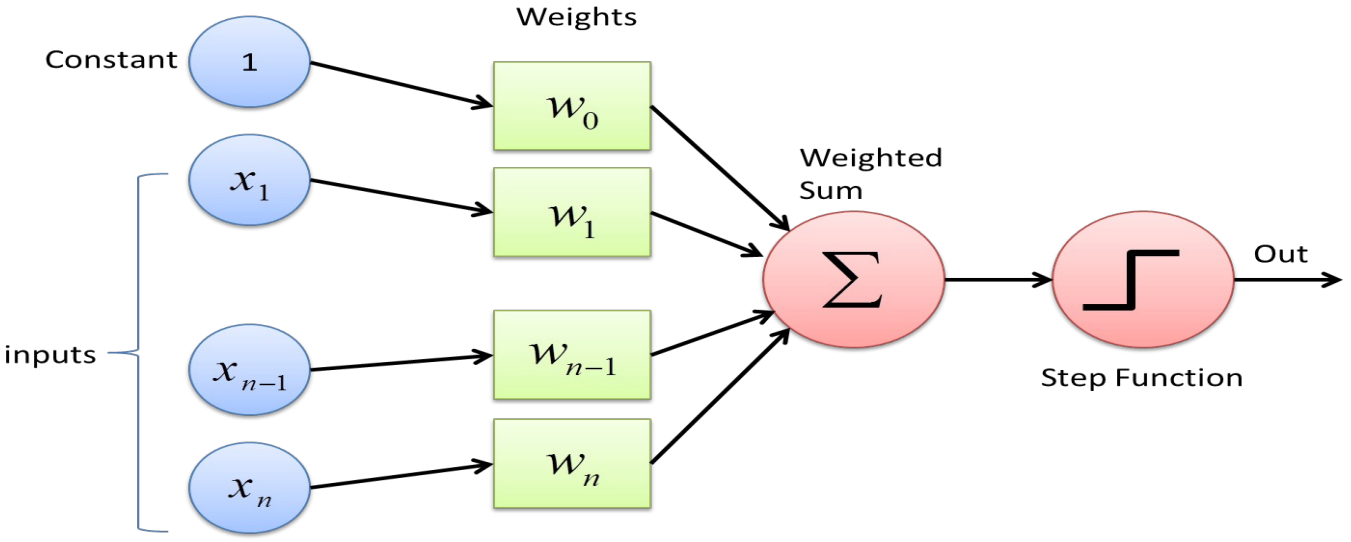
Perceptrons

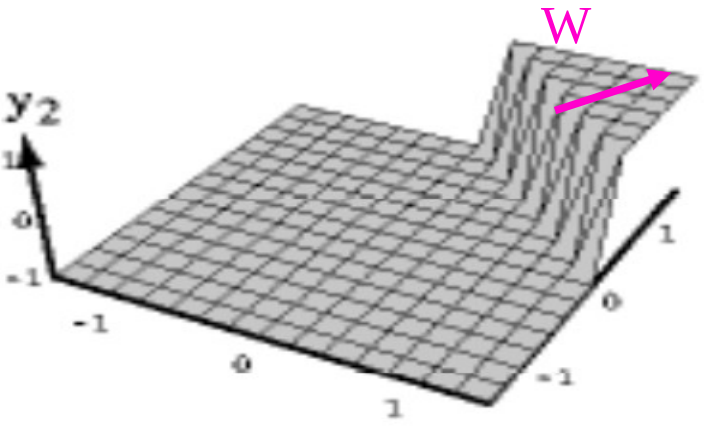
$$o(x_1,x_2\ldots,x_n) = \begin{cases} 1 \quad \text{if } w_0 + w_1 x_1 + \ldots + w_n x_n > 0 \\ -1 \text{ otherwise} \end{cases}$$

$$o(\mathbf{x}) = \text{sgn}(\mathbf{w}.\mathbf{x}) \qquad (i=0 \text{ to } n \; ; \; x_0=1)$$

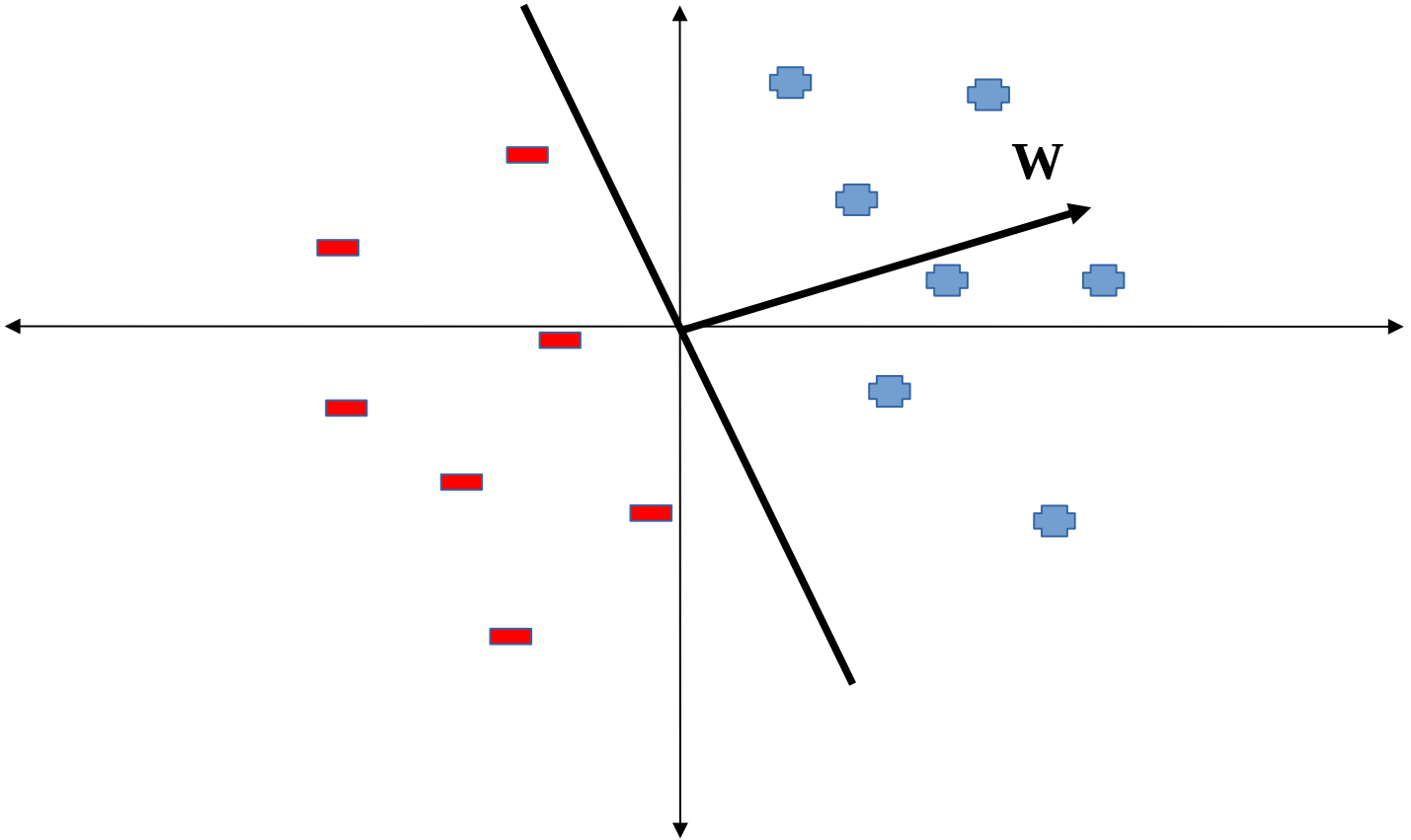
Hypothesis Space: All Hyperplanes $H = \{\mathbf{w} \mid \mathbf{w} \in \Re^{n+1}\}$

Perceptrons

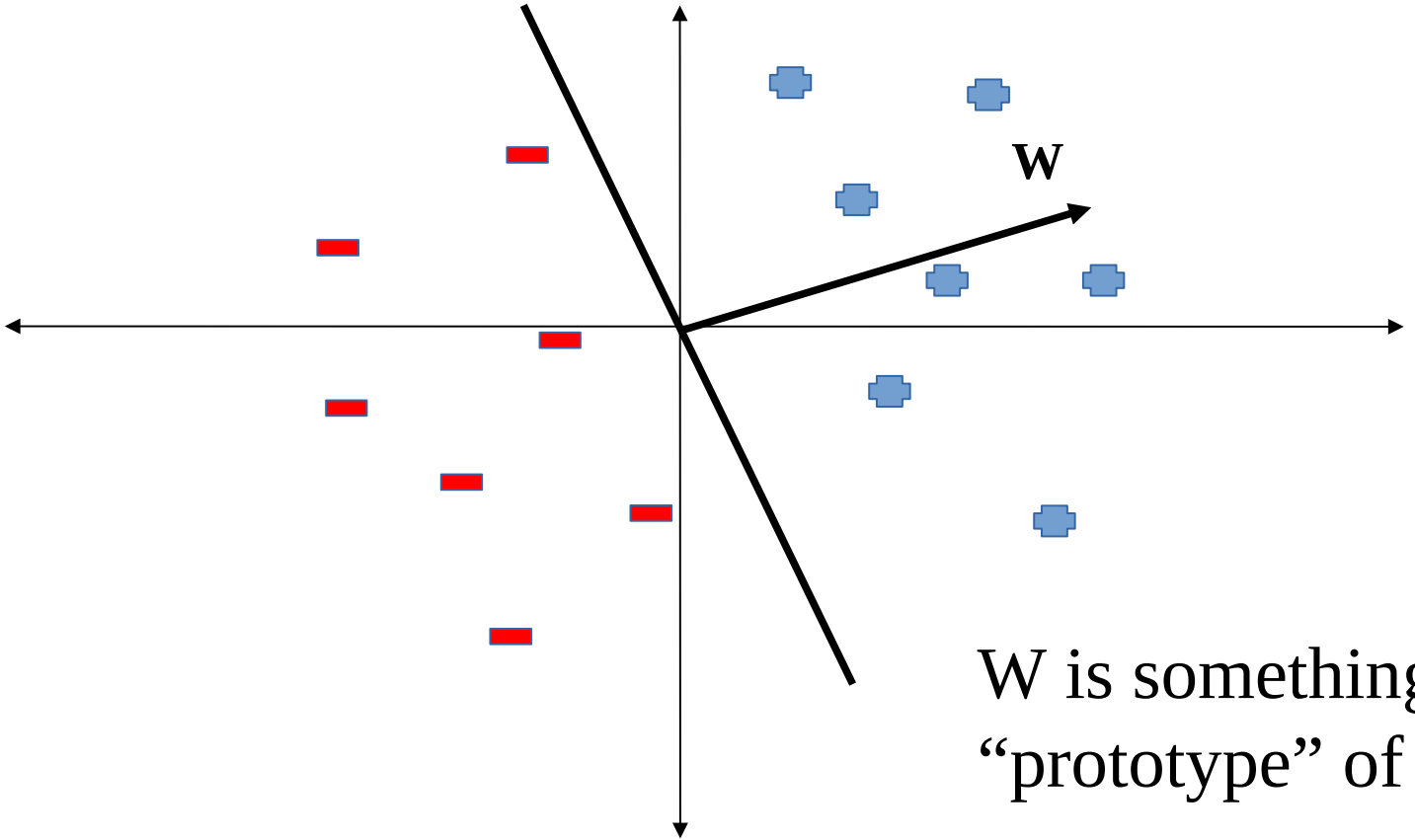




Perceptrons



Perceptrons



W is something like a “prototype” of (+1) class

The Perceptron Training Rule

1. Initialize W at random

2. Iterate over points until convergence:

$$w_i \leftarrow w_i + \Delta w_i$$

$$\Delta w_i = \eta\ (t - o)\ x_i$$

t: target output for the current training example

o: output generated by the perceptron

η: learning rate

The Perceptron Training Rule

1.Initialize W at random

2.Iterate over points until convergence:

$$w_i \leftarrow w_i + \Delta w_i$$

$$\Delta w_i = \eta\ (t - o)\ x_i$$

0 if correct

>0 if +1 and incorrect

<0 if -1 and incorrect

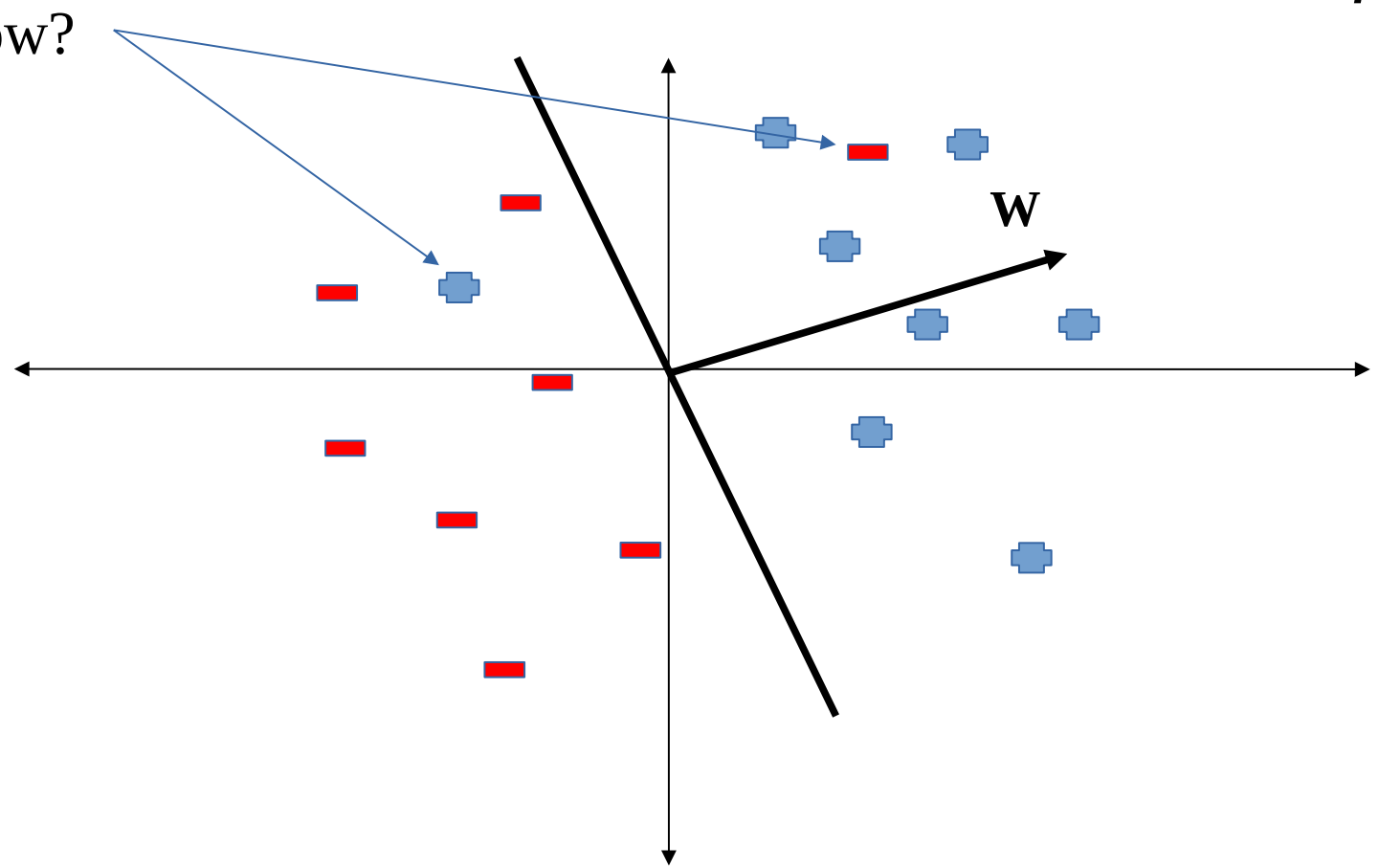
t: target output for the current training example

o: output generated by the perceptron

η: learning rate

Perceptrons

Now?



The Perceptron Training Rule

1. Initialize W at random
2. Iterate over points until convergence:

$$w_i \leftarrow w_i + \Delta w_i$$

$$\Delta w_i = \eta\ (t - o)\ x_i$$

It converges after a finite number of iterations if points are linearly separable

Fails to converge if points are not linearly separable!

Gradient Descent

The perceptron can be considered as a combination of linear + sign functions

Unthresholded perceptrons: $o(\mathbf{x}) = \mathbf{w}.\mathbf{x}$

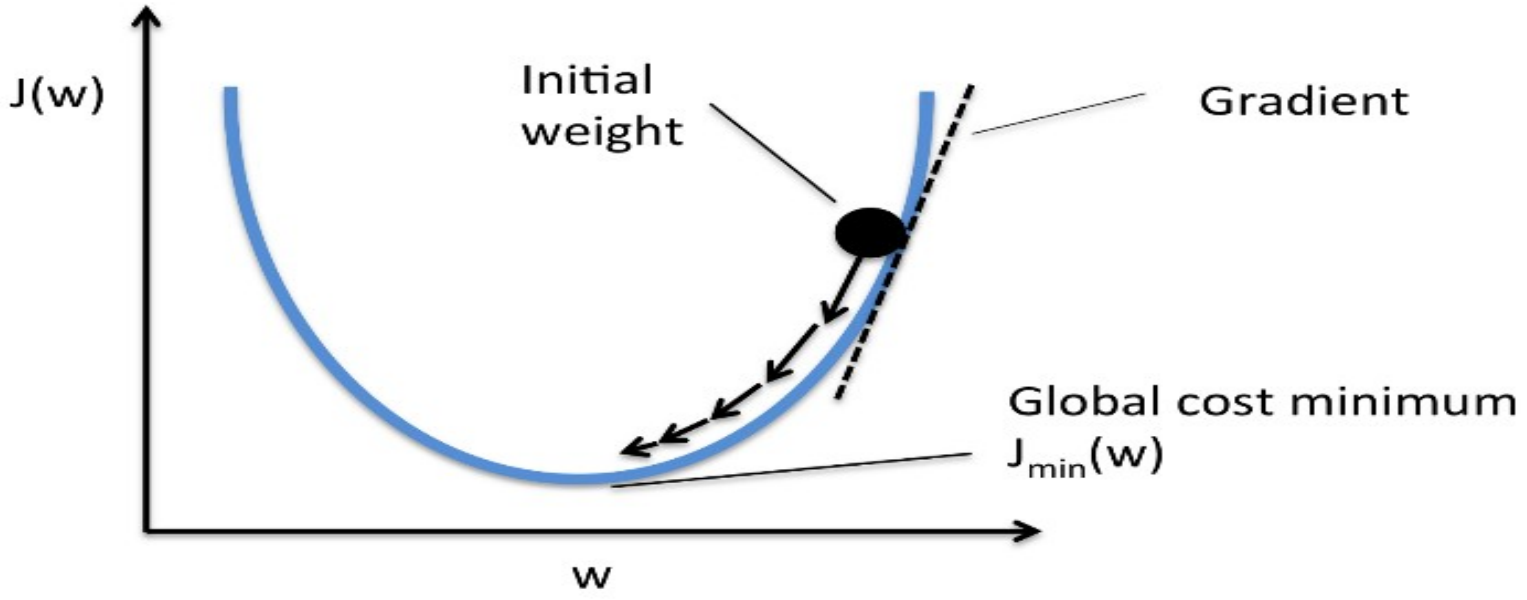
Another strategy:

-define a cost function

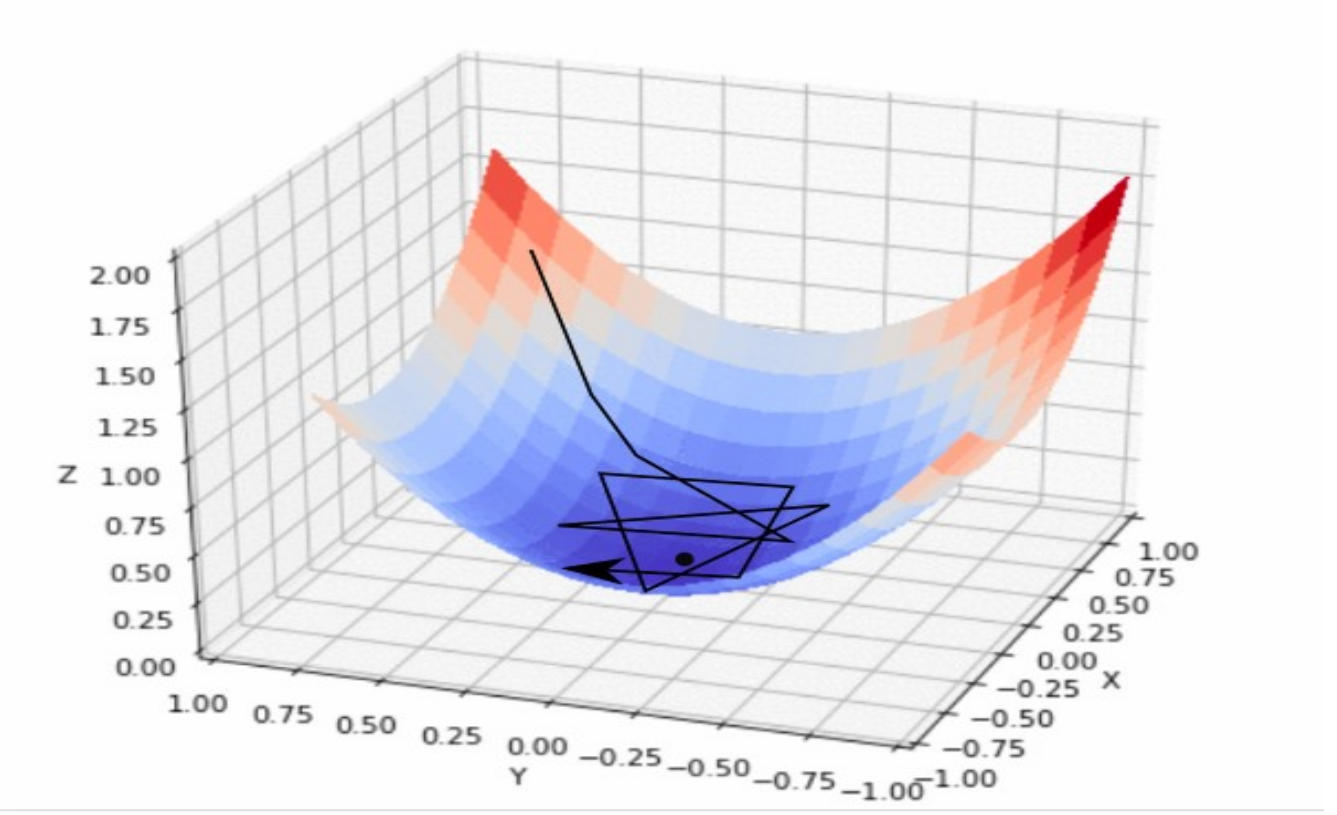
-minimize it

$$E(\mathbf{w}) = \tfrac{1}{2} \sum_D (t-o)^2$$

Gradient Descent



Gradient Descent



Gradient Descent

Unthresholded perceptrons: $o(\mathbf{x}) = \mathbf{w}.\mathbf{x}$

$w_i \leftarrow w_i + \Delta w_i$ $\qquad\qquad$ $\Delta w_i = -\eta\, \partial E/\partial w_i$

$E(\mathbf{w}) = \frac{1}{2} \sum_D (t-o)^2$

$\partial E/\partial w_i = ?$

Gradient Descent

Unthresholded perceptrons: $o(\mathbf{x}) = \mathbf{w}.\mathbf{x}$

$$w_i \leftarrow w_i + \Delta w_i \qquad \Delta w_i = -\eta \, \partial E / \partial w_i$$

$$E(\mathbf{w}) = \tfrac{1}{2} \sum_D (t-o)^2$$

$$\partial E / \partial w_i = \sum_D (t-o)(-x_i)$$

Gradient Descent

GRADIENT-DESCENT($training\_examples, \eta$)

Each training example is a pair of the form $\langle \vec{x}, t \rangle$, where $\vec{x}$ is the vector of input values, and t is the target output value. η is the learning rate (e.g., .05).

- Initialize each w_i to some small random value
- Until the termination condition is met, Do
  - Initialize each Δw_i to zero.
  - For each $\langle \vec{x}, t \rangle$ in $training\_examples$, Do
    - Input the instance $\vec{x}$ to the unit and compute the output o
    - For each linear unit weight w_i, Do

$$\Delta w_i \leftarrow \Delta w_i + \eta(t - o)x_i \qquad \text{(T4.1)}$$

  - For each linear unit weight w_i, Do

$$w_i \leftarrow w_i + \Delta w_i \qquad \text{(T4.2)}$$

Stochastic Gradient Descent

Random order of samples

- For each $\langle \vec{x}, t \rangle$ in *training_examples*, Do
  - Input the instance $\vec{x}$ to the unit and compute the output o
  - For each linear unit weight w_i, Do

$$\Delta w_i \leftarrow \Delta w_i + \eta(t - o)x_i$$

- For each linear unit weight w_i, Do

$$w_i \leftarrow w_i + \Delta w_i$$

Move the update into the inner loop

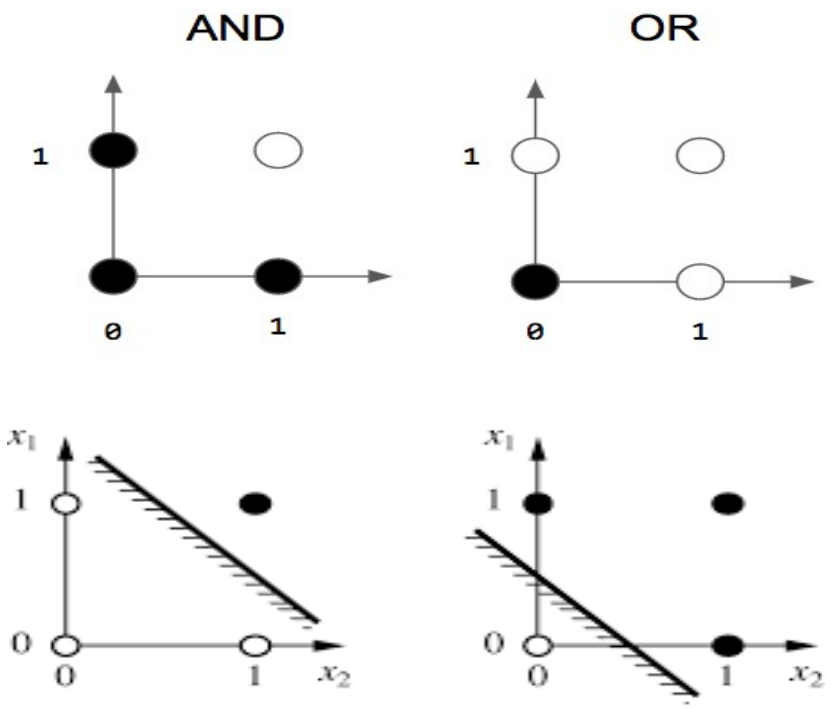
Remarks

- The perceptron training rule converges after a finite number of iterations to a hypothesis that perfectly classifies the data, provided the examples are linearly separable
- The delta rule converges only asymptotically toward the minimum error hypothesis, but regardless of the data linear separability (general approach)

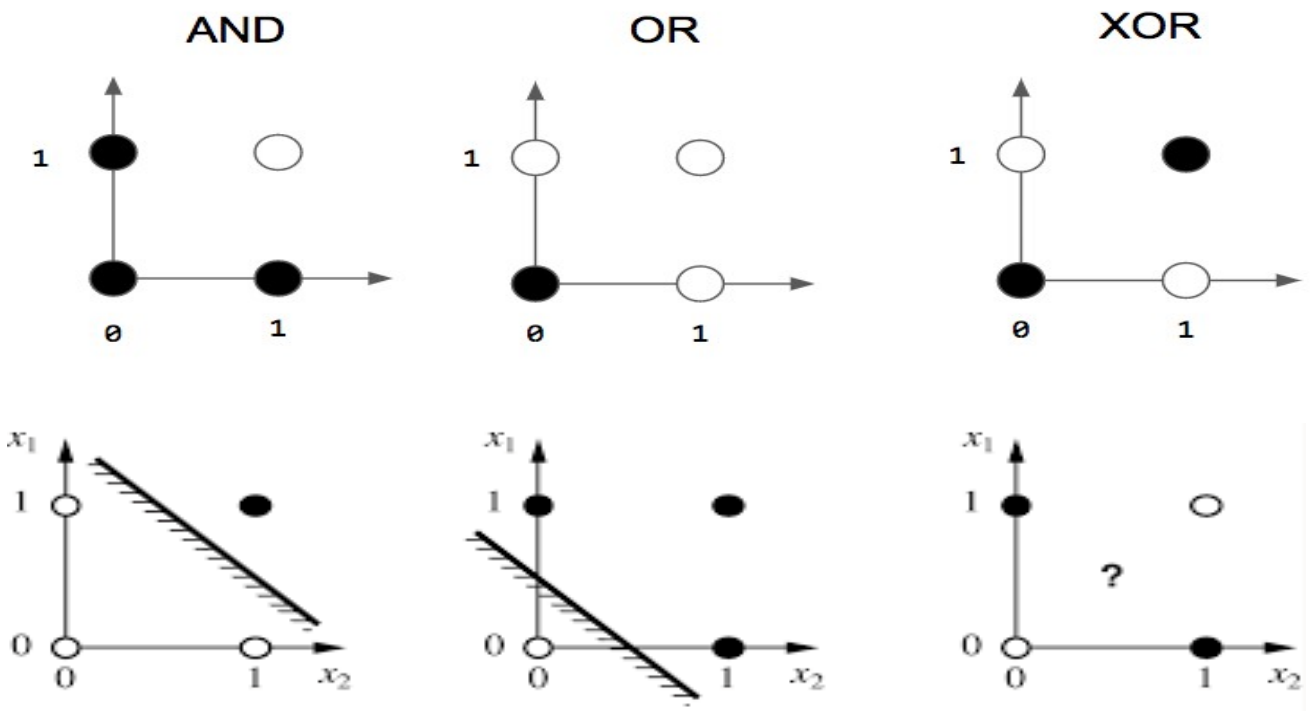
Representational Power

- Perceptrons can represent all the primitive Boolean functions AND, OR, NAND (¬AND) and NOR (¬OR)

Representational Power



The XOR problem



Representational Power

- Perceptrons can represent all the primitive Boolean functions AND, OR, NAND (¬AND) and NOR (¬OR)
- They cannot represent all Boolean functions (for example, XOR)
- Every Boolean function can be represented by some network of perceptrons two levels deep

$$p \oplus q = (p \vee q) \wedge \neg (p \wedge q)$$

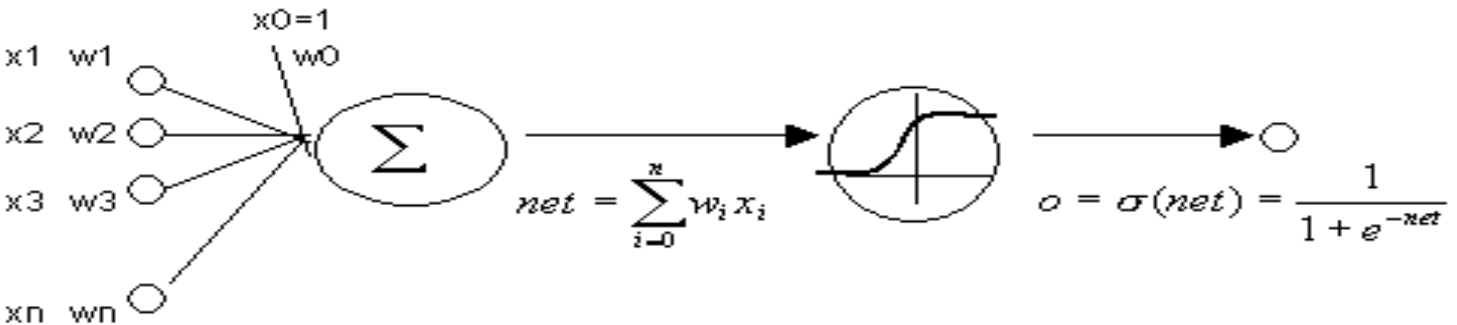
Multilayer Networks

ANNs with two or more layers are able to represent complex nonlinear decision surfaces.

We need nonlinear unit in the hidden layers. (Why?)

Differentiable units can help learning.

Multilayer Networks



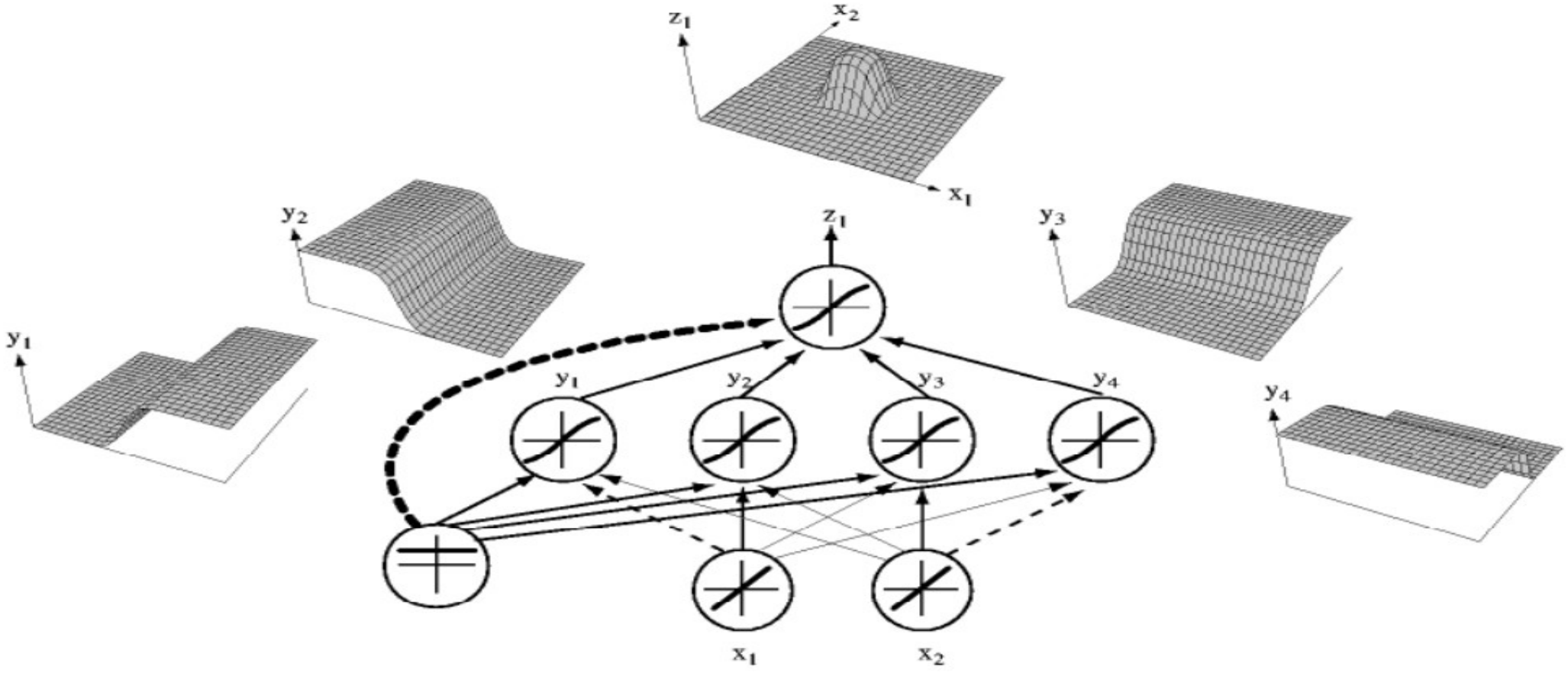
[The sigmoid threshold unit]

- Differentiable Threshold (Sigmoid) Units

$o = \sigma(\mathbf{w}.\mathbf{x})$ $\quad \sigma(y) = 1/(1+e^{-y})$

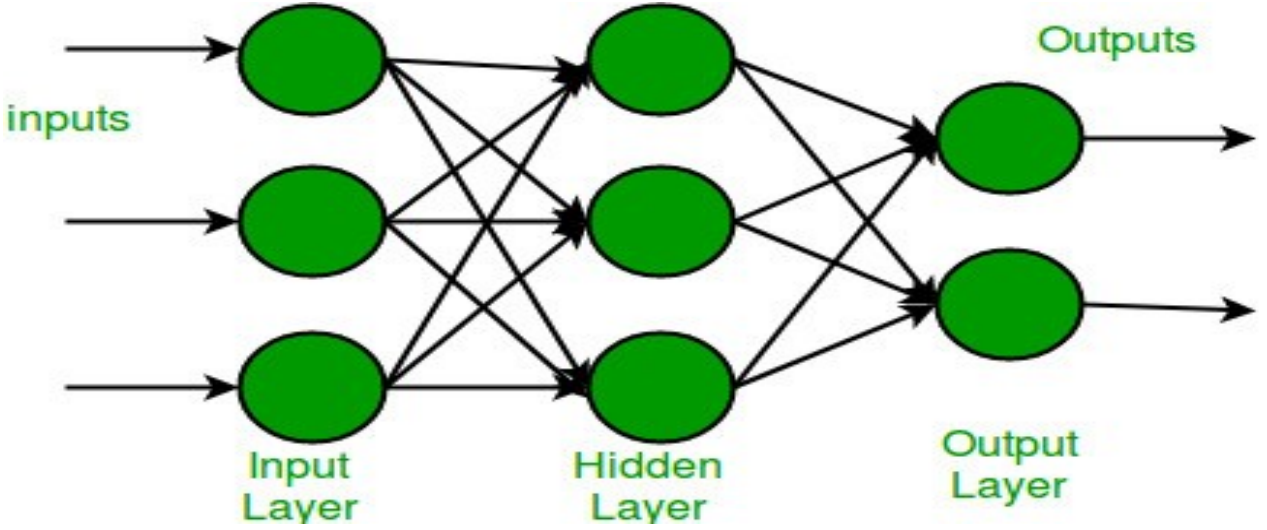
$\partial\sigma/\partial y = \sigma(y)\,[1-\sigma(y)]$

Example



The BackPropagation Algorithm

How to learn the weights in a multilayer perceptron?



The BackPropagation Algorithm

X_{ji} : i-th input to unit j

w_{ji} : weight associated with the i-th input to unit j

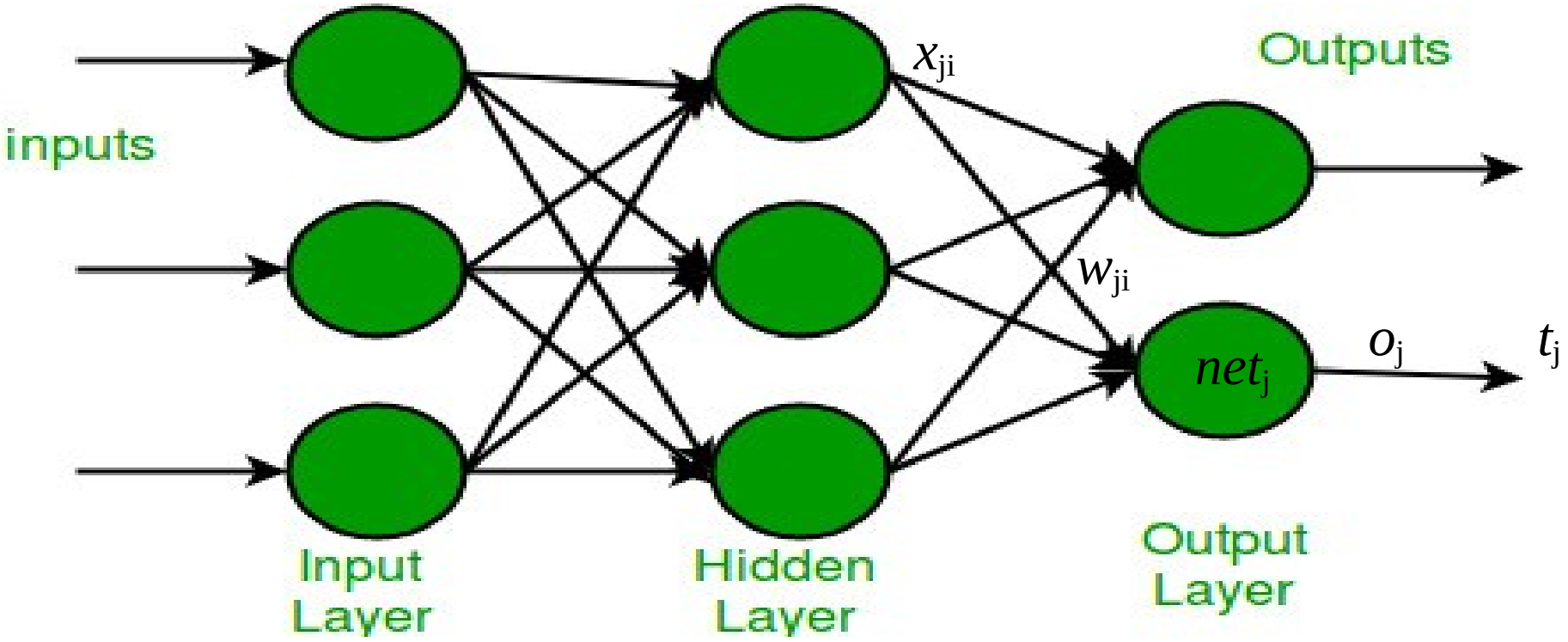
$net_j = \sum_i w_{ji} x_{ji}$: weighted sum of inputs for unit j

o_j : output computed by unit j

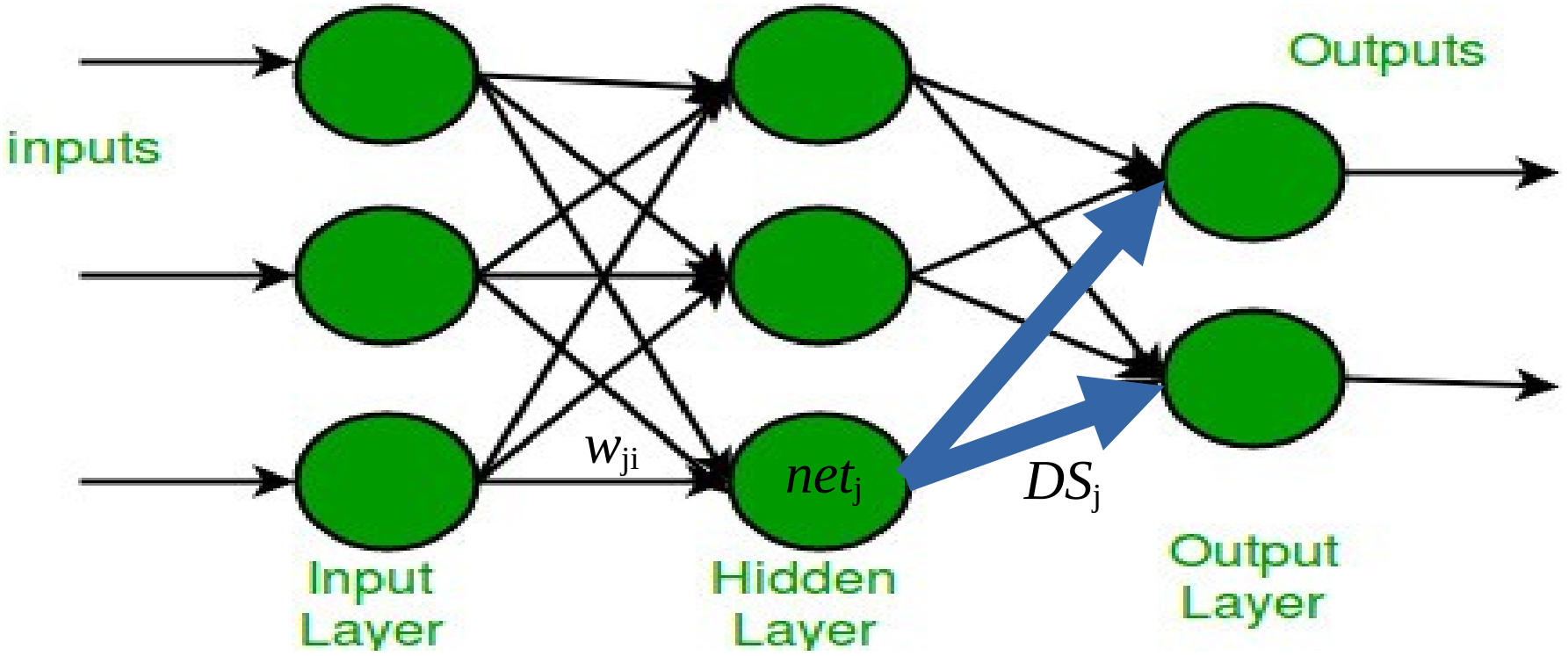
t_j : target output for unit j

DS(j) : DownStream(j), set of units whose inputs include the output of unit j

The BackPropagation Algorithm



The BackPropagation Algorithm



The BackPropagation Algorithm

$o=\sigma(\mathbf{w}.\mathbf{x})$ $\quad \sigma(y)=1/(1+e^{-y})$ $\quad \partial\sigma/\partial y= \sigma(y).[1-\sigma(y)]$

$$E(\boldsymbol{w}) = \tfrac{1}{2} \sum_D \sum_{k\in outputs} (t_k - o_k)^2 = \sum_D E_d$$

$$net_j = \sum_i w_{ji} x_{ji} \quad \rightarrow \quad \partial E_d/\partial w_{ji} = \partial E_d/\partial net_j \,.\, x_{ji}$$

The BackPropagation Algorithm

- Case 1: Output Unit k $\rightarrow o_k = \sigma(net_k) = \sigma(\sum_j w_{kj}.x_j)$

$$\partial E_d/\partial net_k = \partial E_d/\partial o_k \times \partial o_k/\partial net_k \equiv -\delta_k$$

$$\partial E_d/\partial o_k = -(t_k - o_k) \qquad \partial o_k/\partial net_k = o_k(1 - o_k)$$

$$\Rightarrow \quad \Delta w_{kj} = -\eta\, \partial E_d/\partial w_{kj} = \eta\,(t_k - o_k)\, o_k(1 - o_k)\, x_{kj}$$

The BackPropagation Algorithm

- Case 1: Output Unit k

$$\Rightarrow \qquad \Delta w_{kj} = -\eta \, \partial E_d / \partial w_{kj} = \eta \, \underbrace{(t_k - o_k)\, o_k(1 - o_k)}_{\delta_k} \, x_{kj}$$

The BackPropagation Algorithm

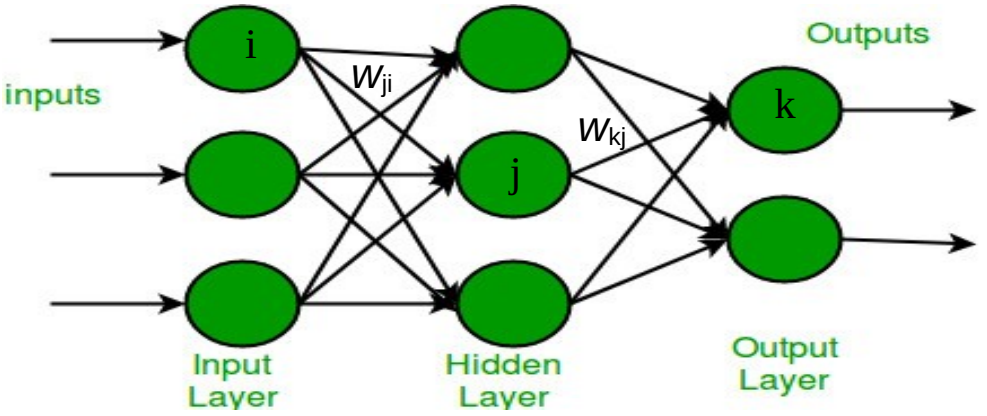
- Case 1: Output Unit k

$$\Rightarrow \quad \Delta w_{kj} = -\eta \, \partial E_d / \partial w_{kj} = \eta \, \underbrace{(t_k - o_k)}_{\text{Error}} \, \underbrace{o_k(1 - o_k)}_{\text{Gradient Of Activation}} \, x_{kj} \rightarrow \text{Input}$$

The BackPropagation Algorithm

- Case 2: Hidden Unit j

$$o_k = \sigma(\mathbf{w}.\mathbf{x}) = \sigma(\sum_j w_{kj}.x_j) = \sigma(\sum_j w_{kj}.(\sigma(\sum_i w_{ji}.x_i)))$$



The BackPropagation Algorithm

– Case 2: Hidden Unit j

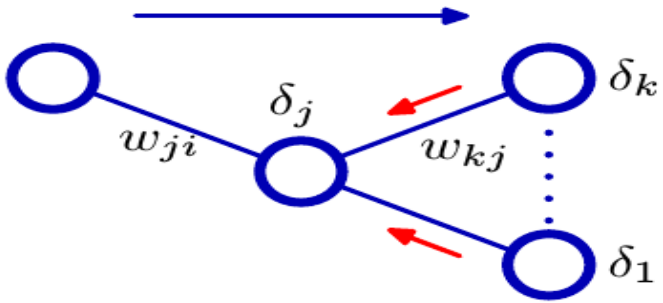
$$\partial E_d/\partial net_j = \sum_{k\in DS(j)} \partial E_d/\partial net_k \times \partial net_k/\partial o_j \times \partial o_j/\partial net_j$$

$$= - \sum_{k\in DS(j)} \delta_k \times \partial net_k/\partial o_j \times \partial o_j/\partial net_j$$

$$= - \sum_{k\in DS(j)} \delta_k w_{kj} o_j (1-o_j)$$

$$\Rightarrow \quad \delta_j = - o_j (1- o_j) \sum_{k\in DS(j)} \delta_k w_{kj}$$

$$\Delta w_{j\,i} = - \eta \, \partial E_d/\partial w_{ji} = \eta \, \delta_j x_{ji}$$

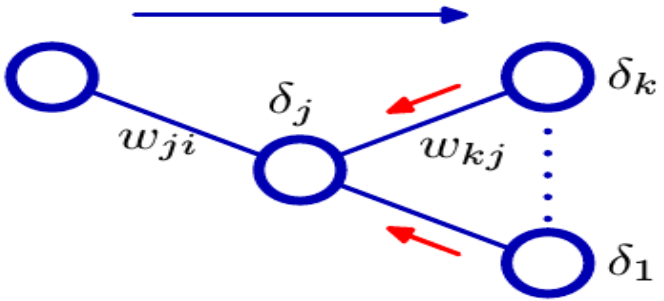


The BackPropagation Algorithm

- Case 2: Hidden Unit j

$$\Rightarrow \quad \delta_j = -\, o_j (1 - o_j) \sum_{k \in DS(j)} \delta_k w_{kj}$$

$$\Delta w_{j\,i} = -\, \eta\, \partial E_d / \partial w_{ji} = \eta\, \delta_j x_{ji}$$



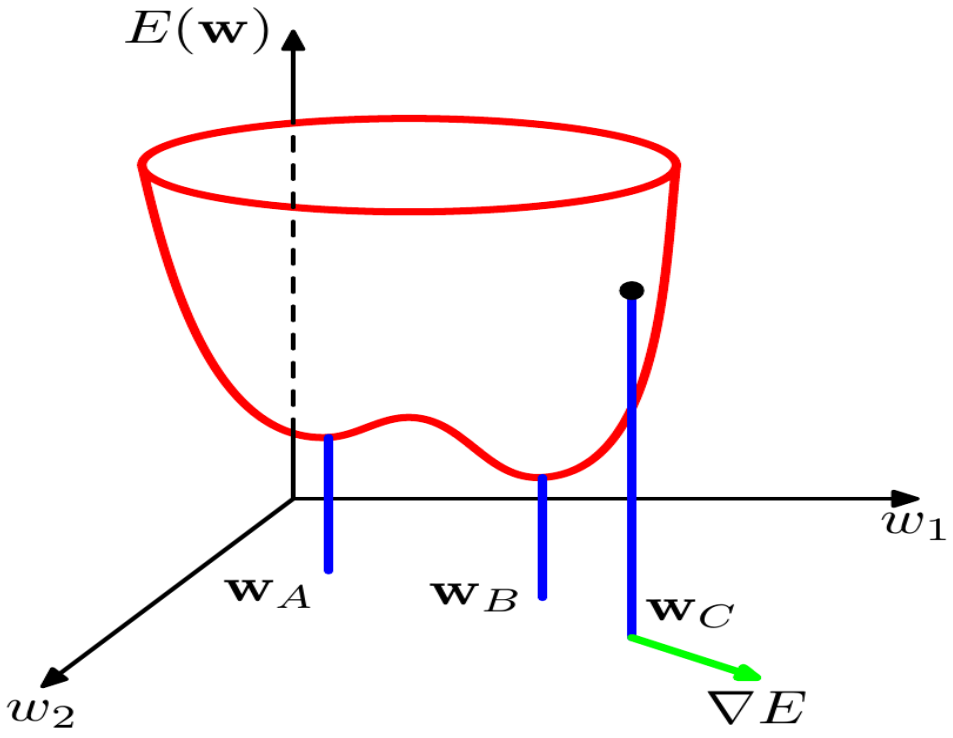
This is valid for any number of hidden layers

Remarks on the BP Algorithm

- Implements a gradient descend search

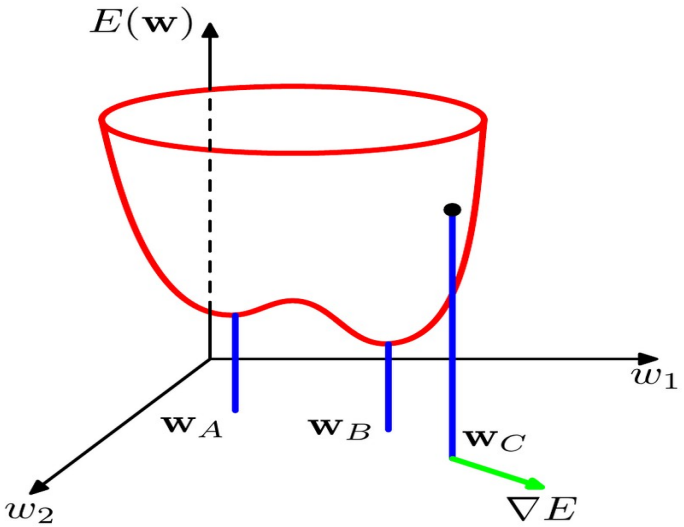
Remarks on the BP Algorithm

- Implements a gradient descend search
- Local minimum



Remarks on the BP Algorithm

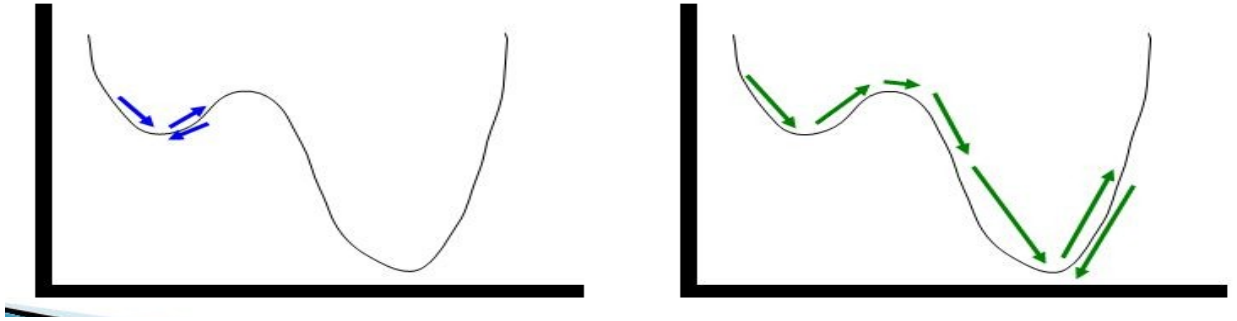
- Implements a gradient descend search
- Heuristics
  - Momentum term
  - Stochastic gradient descent
  - Training multiple networks

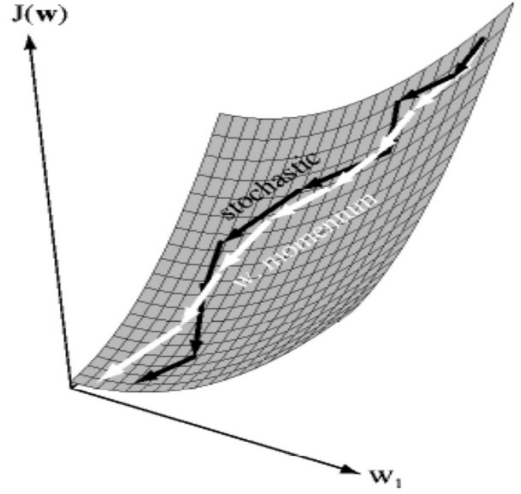


Momentum term

Adds “momentum” to gradient descent

$$\Delta w_{t+1} = - \eta \, \partial E / \partial w_t + \alpha \, \Delta w_t$$





η,α regulate the method → new problem, select parameters

Stochastic gradient descent

GRADIENT-DESCENT(*training_examples*, η)

Each training example is a pair of the form $\langle \vec{x}, t \rangle$, *where* $\vec{x}$ *is the vector of input values, and* t *is the target output value.* η *is the learning rate (e.g., .05).*

- Initialize each w_i to some small random value
- Until the termination condition is met, Do
  - Initialize each Δw_i to zero.
  - For each $\langle \vec{x}, t \rangle$ in *training_examples*, Do
    - Input the instance $\vec{x}$ to the unit and compute the output o
    - For each linear unit weight w_i, Do

$$\Delta w_i \leftarrow \Delta w_i + \eta(t - o)x_i \qquad \text{(T4.1)}$$

  - For each linear unit weight w_i, Do

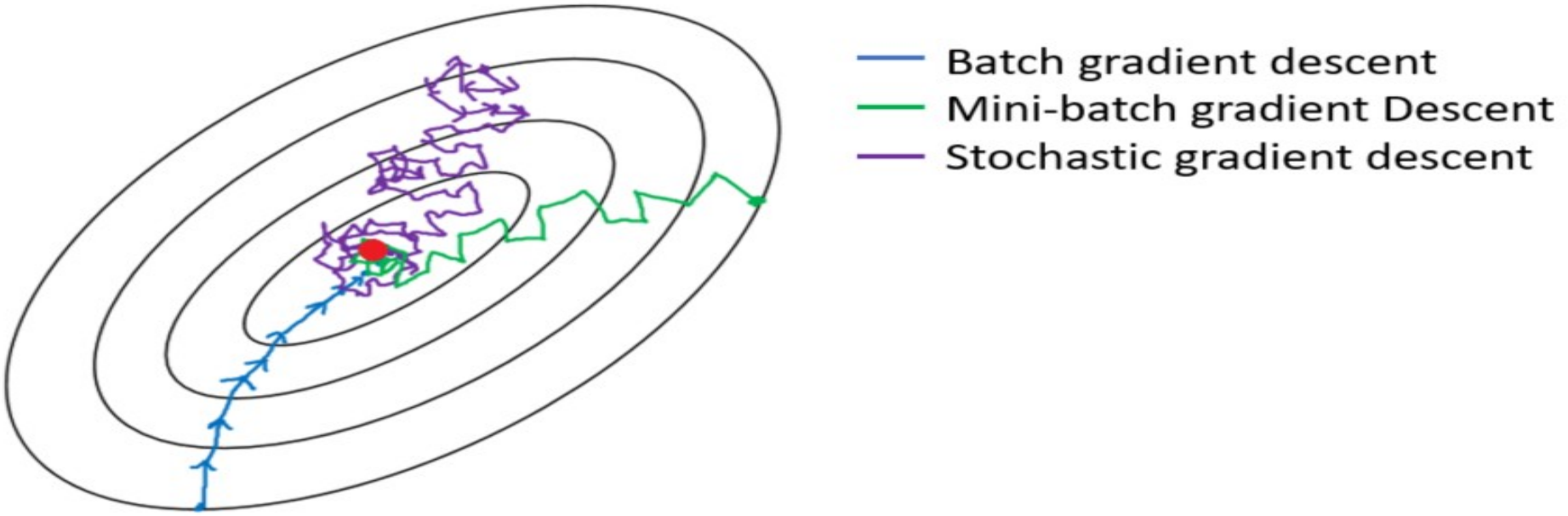
$$w_i \leftarrow w_i + \Delta w_i$$

Move the update into the inner loop

Stochastic gradient descent

- Batch: compute all deltas, then update weights
- Stochastic: for each pattern, compute delta and update weights
- Minibatch: select a small subset of patterns, compute their deltas, then update weights

Stochastic gradient descent



Training multiple networks

- Training is partially random (initial weights, stochastic descent)
- Train multiple networks, keep the best

Representational Power of FeedForward ANNs

- Boolean functions: exactly with two layers and enough hidden neurons
- Continuous functions: bounded functions can be approximated with arbitrarily small error with two layers (sigmoid hidden units and linear output units)
- Arbitrary functions: can be approximated to arbitrary accuracy with three layers (two hidden layers with sigmoid units plus linear output units)

Theorems by Cybenko (88, 89)

Representational Power of FeedForward ANNs

- Hypothesis Space search and inductive Bias

  Hypothesis Space:

Representational Power of FeedForward ANNs

- Hypothesis Space search and inductive Bias

  Hypothesis Space:

  Complex non-linear function of the inputs, dependent of a set of network weights

  or simply: *n*-dimensional Euclidean space of network weights

Representational Power of FeedForward ANNs

- Hypothesis Space search and inductive Bias

  Hypothesis Space: *n*-dimensional Euclidean space of network weights

  Inductive Bias:

Representational Power of FeedForward ANNs

- Hypothesis Space search and inductive Bias

  Hypothesis Space: *n*-dimensional Euclidean space of network weights

  Inductive Bias: Smooth interpolation between data points

Representational Power of FeedForward ANNs

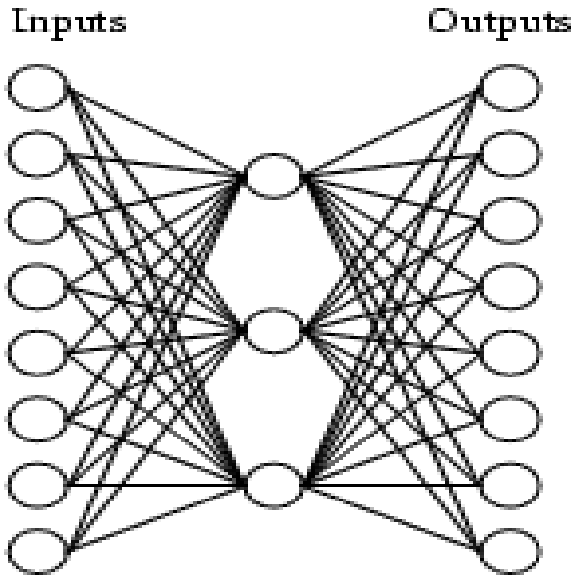
- Hypothesis Space search and inductive Bias
  - Hypothesis Space: *n*-dimensional Euclidean space of network weights
  - Inductive Bias: Smooth interpolation between data points
- Trees: Learning as search over a discrete hypothesis space.
- ANNs: Learning as cost minimization over continuous parametric functions.

Both: highly expressive hypothesis space, simple to complex search

Representational Power of FeedForward ANNs

- Hidden Layer Representation
  - Encoding of information
  - Discovering of new features not explicit in the input representation

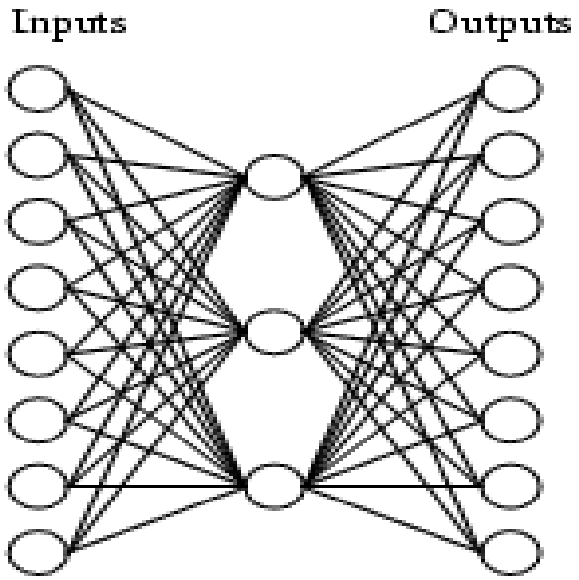
Hidden representation



Auto encoder

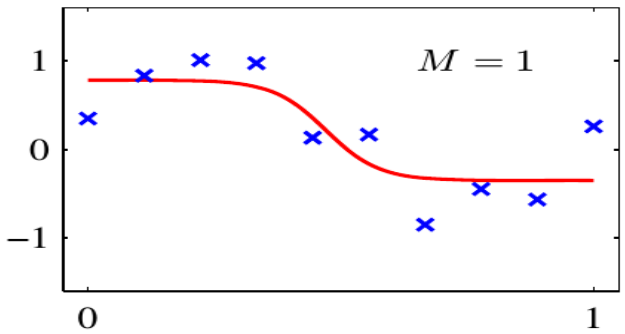
Input		Output
10000000	→	10000000
01000000	→	01000000
00100000	→	00100000
00010000	→	00010000
00001000	→	00001000
00000100	→	00000100
00000010	→	00000010
00000001	→	00000001

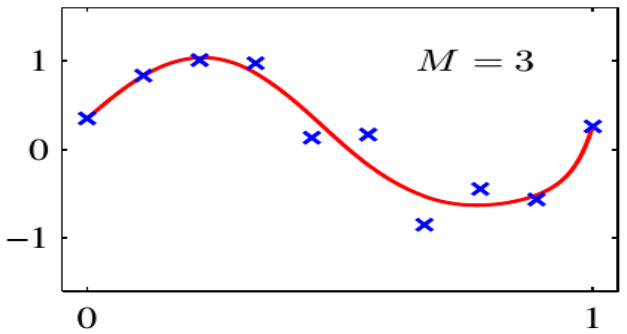
Hidden representation

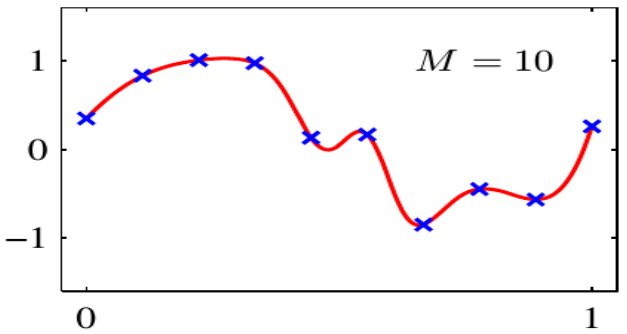


Input		Hidden Values				Output
10000000	→	.89	.04	.08	→	10000000
01000000	→	.15	.99	.99	→	01000000
00100000	→	.01	.97	.27	→	00100000
00010000	→	.99	.97	.71	→	00010000
00001000	→	.03	.05	.02	→	00001000
00000100	→	.01	.11	.88	→	00000100
00000010	→	.80	.01	.98	→	00000010
00000001	→	.60	.94	.01	→	00000001

Representational Power

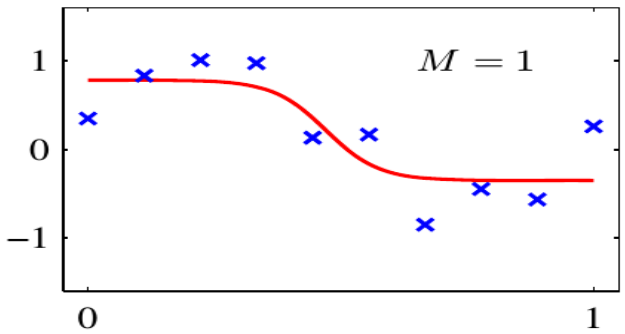


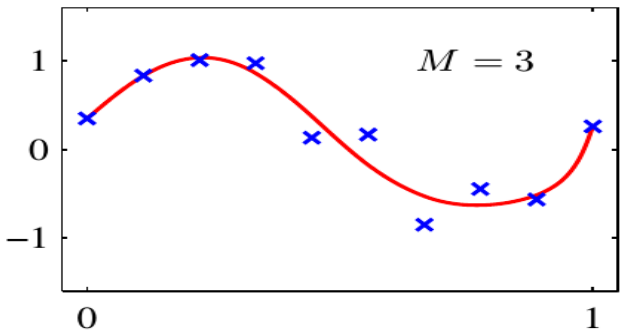


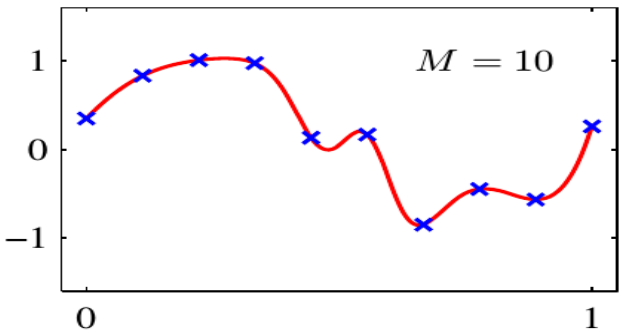


M = # of hidden units

Representational Power





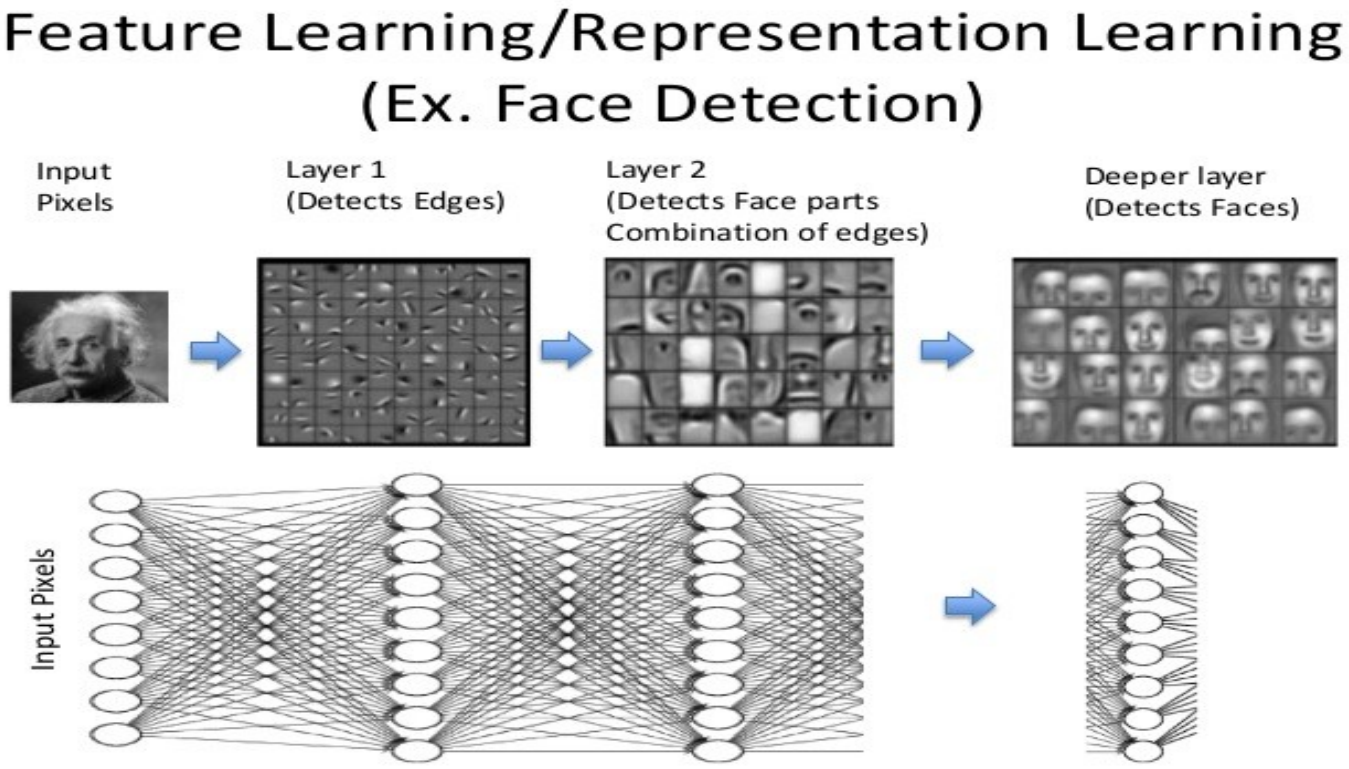


M = # of hidden units

More hidden units, more capability. More flexibility.

We can fit any function. We can fit noise!

Hidden representation



Generalization, Overfitting and Stopping Criterion

As in DTL, training increases complexity and risks overfitting

Is it wise to train the network until the minimum error?

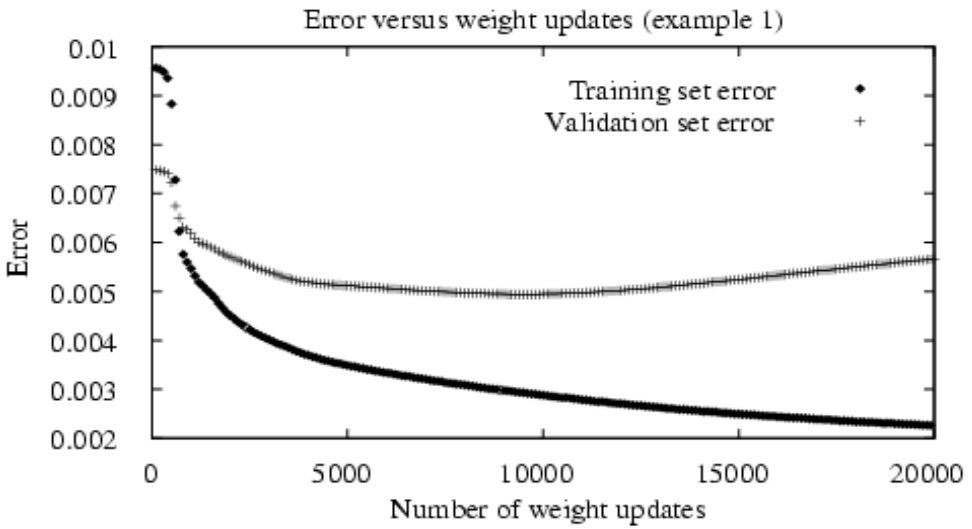
Generalization, Overfitting and Stopping Criterion

As in DTL, training increases complexity and risks overfitting

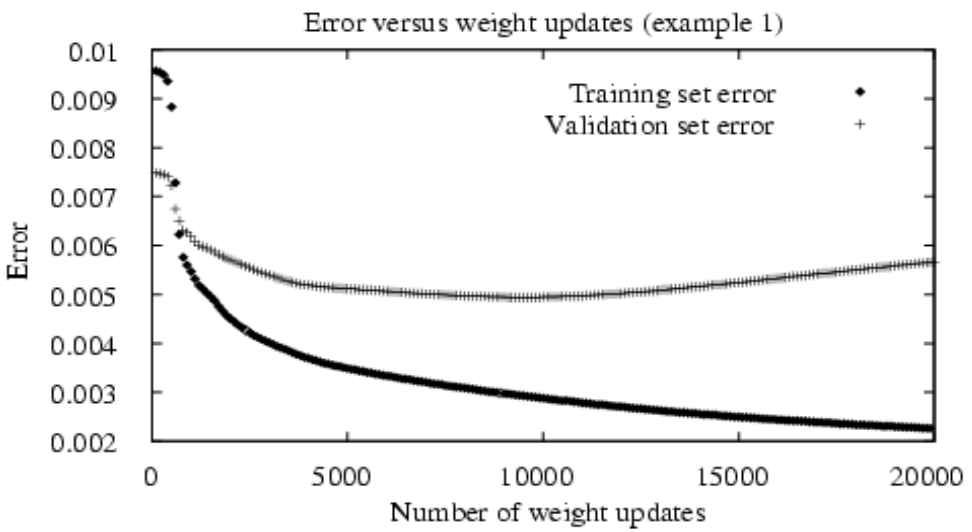
What is an appropriate condition for terminating the weight update loop?

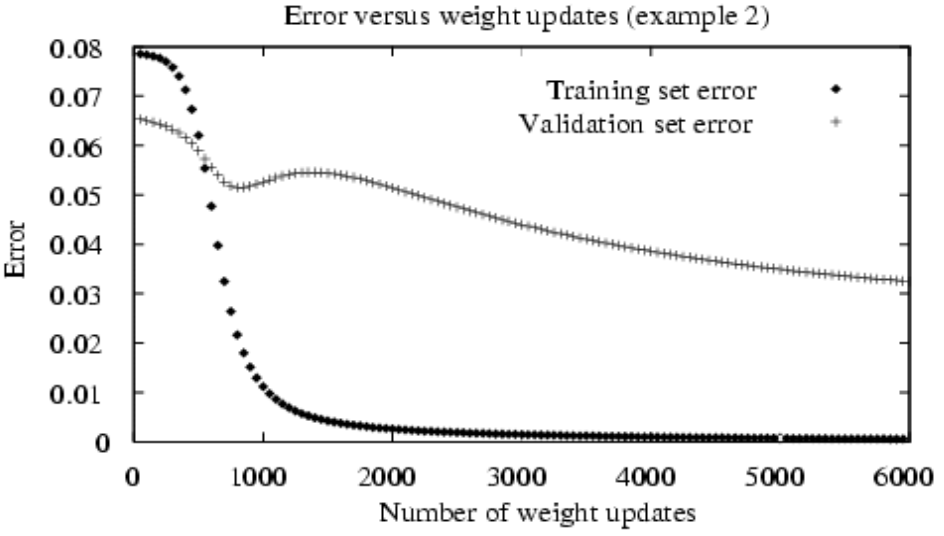
- Hold-out Validation
- *k*-fold Cross Validation

Overfitting in ANNs



Overfitting in ANNs

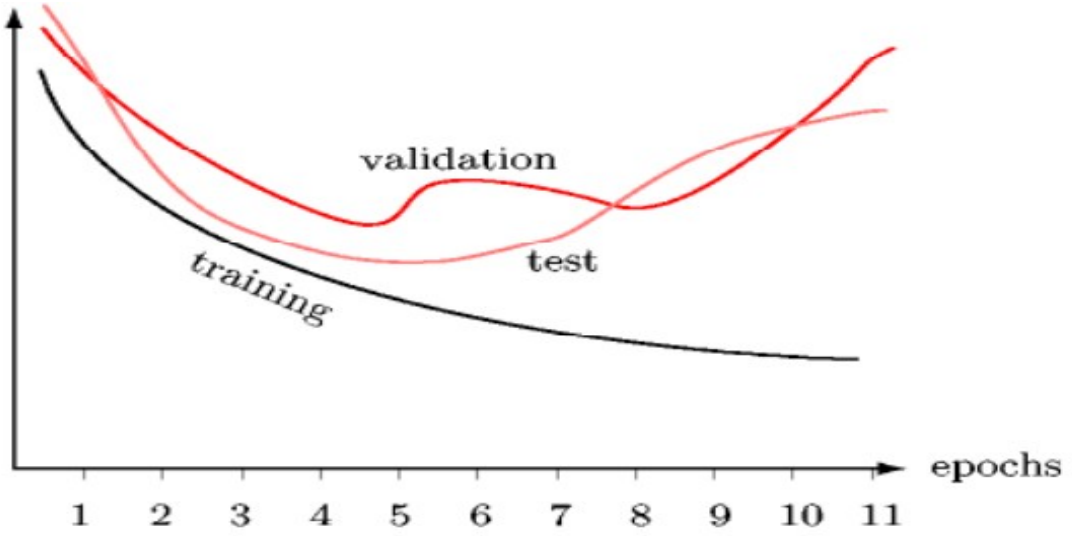




Overfitting in ANNs

Early stopping:

- Use a validation set to monitor generalization error
- Stop at the validation minimum



Regularization

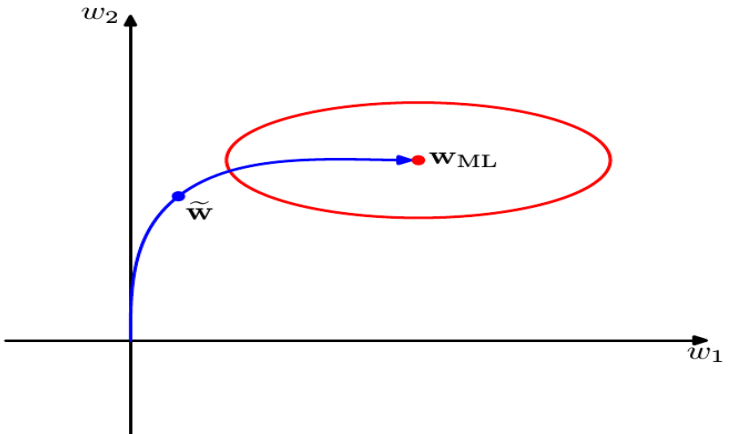
E = cost + complexity

Regularization

E = cost + complexity

Weight Decay:

$$E(\boldsymbol{w}) = \tfrac{1}{2} \sum_D \sum_{k \in \text{outputs}} (t_k - o_k)^2 + \gamma \sum_{ij} (w_{ji})^2$$



Small weights produce simpler solutions

Keeping weights small prevents overfitting

Example: Face Recognition

- The Task
  - Classifying camera images of faces of 20 different people, including 32 images per person, varying the person’s expression (happy, sad, angry, neutral), the direction in which they are looking (left, right, straight ahead, up), and whether or not they are wearing sunglasses
  - There are also variation in the background behind the person, the clothing worn by the person and the position of the face within the image

Example: Face Recognition

- Each image has a 120x128 resolution, with pixels in a greyscale intensity from 0 (black) to 255 (white)

Task: Learning the direction in which the person is facing

– Design Choices
  - Input encoding: 30x32 coarse intensity values
  - Output encoding: 4 distinct output units

Example: Face Recognition

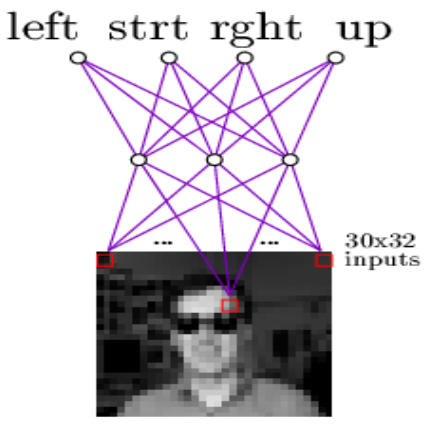
- Network structure: i : h: o

  i = 30 x 32 h = 3 to 30 o = 4

- Learning parameters:

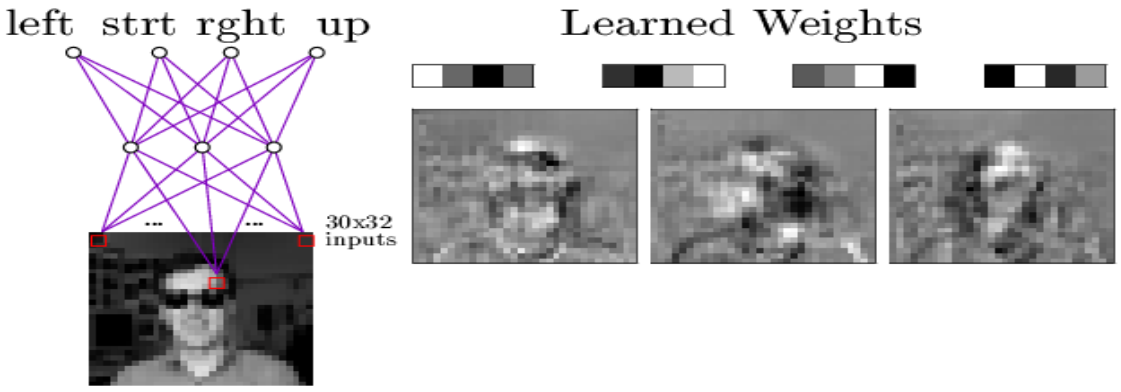
  learning rate = 0.3 momentum = 0.9

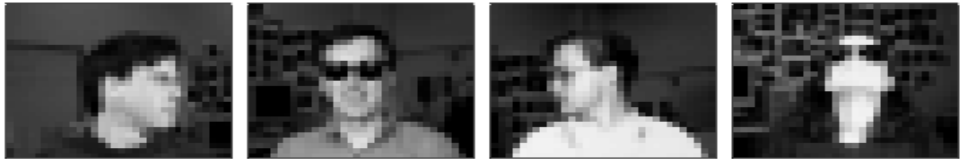
Example: Face Recognition



Typical input images

Example: Face Recognition





Typical input images

Recap

- ANNs: Learning as cost minimization over continuous parametric functions.
- Gradient descent, backpropagation
- Overfitting
- Learning of an internal representation
- New problem: set model hyper-parameters

ANNs in practice: classification

- We discussed how to learn a continuous output. What about discrete outputs (classes)? Ideas?

ANNs in practice: classification

- We can learn discrete outputs using continuous outputs and some discretization rule.
- There are better ways to learn classes
- For C classes we use C output units (1-of-C)
- We want each unit to be 1 if the pattern belongs to that class, 0 otherwise
- In the real world, the output should be the probability of the class

ANNs in practice: classification

- In the real world, the output should be the probability of the class
- To learn probabilities, we use output units that can be interpreted as probabilities (positives and sum to one)
- Typically we use SOFTMAX:

$$O_i(X) = e^{Z_i} / \sum e^{Z_k}$$

$$Z_i = net_i$$

ANNs in practice: classification

- With SOFTMAX we use a different loss function:

$$loss\left(O\left(X\right),t\right)=-\sum 1_{y=c}\log\left(O\left(X\right)_c\right)=-\log\left(O\left(X\right)_c\right)$$